

- Question: Why does defining a custom constructor suppress the default constructor in C#?

In C#, when you **define any custom constructor**, the compiler **does not automatically create a default (parameterless) constructor**.

Why:

- The compiler assumes you want **full control** over how objects are initialized.
 - If it still provided a default constructor, it could **conflict with your custom logic**.
- Question: How does method overloading improve code readability and reusability?

Readability: You don't need separate confusing method names like AddTwoNumbers or AddThreeNumbers. One name (Sum) is enough.

Reusability: You can handle different types or numbers of parameters without rewriting new methods.

Intuitive code: The compiler automatically chooses the correct method based on the arguments you pass.

- Question: What is the purpose of constructor chaining in inheritance?

It **calls the parent constructor** automatically from the child constructor

Ensures all properties (base + derived) are **properly set**

Helps **avoid code duplication** and keeps initialization **organized and safe**

- Question: How does new differ from override in method overriding?

new keyword

- Hides the base class method.
- The method called depends on the **reference type**, not the object type.
- **Parent reference** calls the **base method**, **Child reference** calls the **new method**.

override keyword

- Replaces the base class method.
- The method called depends on the **object type**, regardless of the reference type.
- Ensures **polymorphic behavior**: even a Parent reference to a Child object calls the Child's method
 - Question: Why is ToString() often overridden in custom classes?

ToString() is often overridden in custom classes to provide a **meaningful and readable representation** of the object instead of the default, which just shows the class name.

- Question: Why can't you create an instance of an interface directly?

You **can't create an instance of an interface directly** because an interface only defines **what a class should do**, not **how it does it**.

- It has **no actual code or data**, just method/property signatures.
- Only **a class that implements the interface** provides the real implementation.
 - Question: What are the benefits of default implementations in interfaces introduced in C# 8.0?

Default implementations in interfaces (C# 8.0) provide several benefits:

1. **Code Reuse** – You can write common functionality once in the interface, and all implementing classes can use it without rewriting.
2. **Backward Compatibility** – You can add new methods to an existing interface **without breaking** existing classes that implement it.
3. **Simpler Design** – Reduces the need for helper or abstract base classes just to provide shared behavior.
4. **Flexibility** – Implementing classes can still **override the default method** if they need custom behavior.

- Question: Why is it useful to use an interface reference to access implementing class methods?

Using an **interface reference** to access an implementing class is useful because it allows your code to **work with any class that implements the interface**, not just one specific class.

Benefits in simple terms:

1. **Polymorphism:** The same code can call Move() on a Car, Bike, or any IMovable object.
2. **Flexibility:** You can change the underlying class without changing the code that uses the interface.
3. **Decoupling:** Your code depends on **what an object can do** (the interface) rather than **how it does it** (the class).

- Question: How does C# overcome the limitation of single inheritance with interfaces?

C# allows a class to **implement multiple interfaces**, which overcomes the **single inheritance limitation** of classes.

In simple terms:

- A class can inherit from **only one base class**.
- But it can **implement many interfaces**, so it can take on **multiple sets of behaviors**.
- Interfaces provide a way to **reuse functionality contracts** without needing multiple class inheritance.

- Question: What is the difference between a virtual method and an abstract method in C#?

🔍 Virtual method

- Has a **default implementation** in the base class.
- Derived classes **can override it**, but don't have to.
- Example: `public virtual void Draw() { ... }`

🔍 Abstract method

- Has **no implementation** in the base class.
- Derived classes **must override and implement it**.
- Example: `public abstract double CalculateArea();`

What is the difference between class and struct in C#?

A **class** in C# is a reference type, which means its objects are stored on the heap and variables hold a reference to the object. Classes support inheritance, can have parameterless or parameterized constructors, and can be null. Changes to a class object through a variable affect the original object.

A **struct** is a value type, so its data is stored directly on the stack (or inline in arrays), and variables hold the actual data. Structs cannot inherit from other structs or classes, cannot have a parameterless constructor, and cannot be null unless wrapped in `Nullable<T>`. Passing a struct to a method copies its data, so changes inside the method don't affect the original unless passed with `ref`.

Structs are best for small, lightweight objects, while classes are suited for more complex, reference-based objects.

If inheritance is relation between classes clarify other relations between classes

? Association

- A class uses or knows about another class.
- Example: A Car class has an Engine object.
- Can be one-to-one, one-to-many, or many-to-many.

? Aggregation

- A special kind of association where one class owns another class, but the owned object can exist independently.
- Example: A Library class has Book objects. Books can exist outside the library too.

? Composition

- A strong form of aggregation where the lifetime of the owned object depends on the owner.
- Example: A House class has Room objects. If the house is destroyed, the rooms are destroyed too.

❓ Dependency

- One class temporarily uses another class for some operation.
- Example: A Printer class uses a Document object to print but doesn't store it.

❓ Interface Implementation

- A class implements an interface, agreeing to provide specific behavior.
- Example: Car implements IMovable to provide Move() functionality.

what is static and dynamic binding

Static Binding (Early Binding)

- Happens **at compile time**.
- The compiler knows **exactly which method or property** will be called.
- Usually occurs with **non-virtual methods, private methods, or static methods**.
- **Fast**, but cannot take advantage of polymorphism.

Dynamic Binding (Late Binding)

- Happens **at runtime**.
- The **method to call is determined by the actual object type**, not the reference type.
- Usually occurs with **virtual methods, overridden methods, and interface methods**.
- Enables **polymorphism**.

